Experiment 1: Implement Linear and Logistic Regression on real-world datasets

Aim

To implement Linear Regression and Logistic Regression on the Bank Marketing dataset.

Theory

1. Dataset Source

Dataset Name: Bank Marketing Dataset
Source: Kaggle
Link: https://www.kaggle.com/datasets/janiobachmann/bank-marketing-dataset

The dataset is related to direct marketing campaigns (phone calls) of a Portuguese banking institution.

2. Dataset Description

The dataset contains information about clients contacted during a marketing campaign and whether they subscribed to a term deposit.

Target Variable

- **y** → Whether the client subscribed to a term deposit
  - "yes" → 1
  - "no" → 0
  -

Dataset Size

- Approximately 45,000+ rows
- 17 input features
- 1 target variable

Characteristics

- Mixture of numerical and categorical features
- Imbalanced dataset (more "no" than "yes")

- Real-world marketing dataset
- Suitable for classification problems

3. Mathematical Formulation of the Algorithms

A. Linear Regression

Linear Regression predicts a continuous output.

Hypothesis Function:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Where:

- β_0 = Intercept
- β_i = Coefficients
- x_i = Features

Cost Function (Mean Squared Error):

$$J(\beta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Objective: Minimize the Mean Squared Error.

B. Logistic Regression

Logistic Regression is used for binary classification.

Hypothesis Function (Sigmoid Function):

$$h(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n)}}$$

Decision Rule:

$$\hat{y} = \begin{cases} 1 & \text{if } h(x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Loss Function (Binary Cross Entropy):

$$J(\beta) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log(h(x_i)) + (1 - y_i) \log(1 - h(x_i))]$$

4. Algorithm Limitations

Linear Regression

- Not suitable for classification problems
- Sensitive to outliers
- Assumes linear relationship between features and target
- Predictions are not restricted between 0 and 1

Logistic Regression

- Assumes linear decision boundary
- Performs poorly when classes are highly overlapping
- Struggles with non-linear datasets
- Sensitive to multicollinearity
- Can be affected by class imbalance

5. Methodology / Workflow

Workflow Steps

Dataset Collection
→ Data Cleaning
→ Handling Missing Values
→ Encoding Categorical Variables
→ Feature Scaling
→ Train-Test Split
→ Model Training (Linear and Logistic Regression)

→ Prediction
→ Performance Evaluation
→ Hyperparameter Tuning

Steps Performed

1. Loaded the dataset.
2. Converted the target variable to numeric form.
3. Applied one-hot encoding for categorical variables.
4. Scaled features using StandardScaler.
5. Split data into training (80%) and testing (20%) sets.
6. Trained Linear Regression and Logistic Regression models.
7. Evaluated models using appropriate metrics.
8. Performed hyperparameter tuning using GridSearchCV.

6. Performance Analysis

A. Linear Regression

Evaluation Metrics Used:

- Mean Squared Error (MSE)
- R^2 Score

Interpretation:

- Lower MSE indicates better performance.
- R^2 closer to 1 indicates better model fit.

Since the target variable is binary, Linear Regression is not ideal but implemented for comparison purposes.

B. Logistic Regression

Evaluation Metrics Used:

- Accuracy
- Precision
- Recall
- Confusion Matrix

Confusion Matrix Interpretation:

	No	
	Predicted	
Predicted	**Yes**	

Actual No	TN	FP
Actual Yes	FN	TP

- Accuracy measures overall correctness.
- Precision measures correctness of positive predictions.
- Recall measures ability to identify actual positives.

7. Hyperparameter Tuning

Logistic Regression Parameters Tuned

Parameter	Values Tested
C	0.01, 0.1, 1, 10
penalty	l1, l2
solver	liblinear

Effect of Tuning:

- Optimal C improves generalization.
- Regularization prevents overfitting.
- Improved balance between precision and recall.
- Slight improvement in overall accuracy.

CODE:

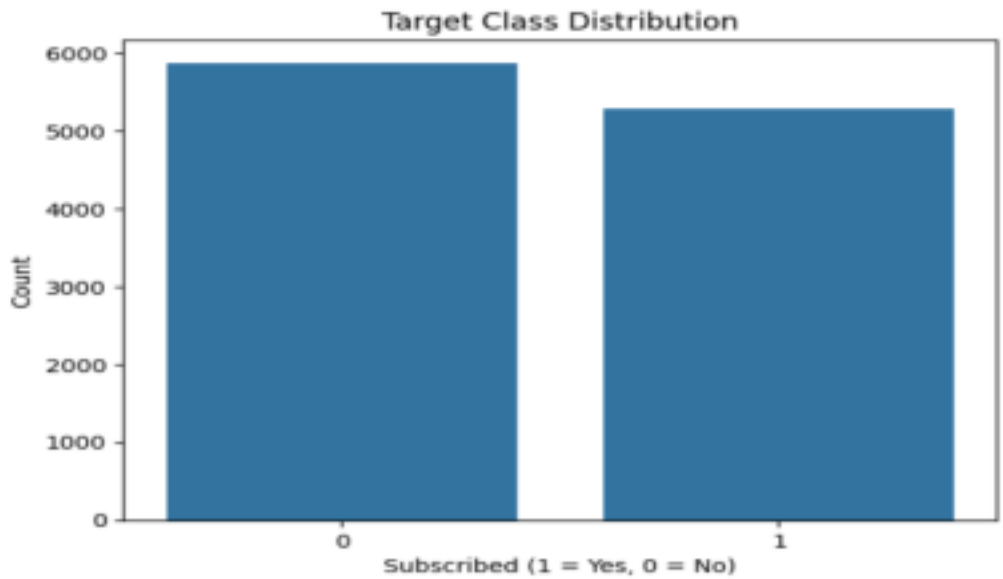
```
# -------- Upload Dataset --------
from google.colab import files
import pandas as pd
import numpy as np
import io

uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_csv(io.BytesIO(uploaded[file_name]))

print("Dataset Loaded Successfully")
print("Columns:", df.columns)


# -------- Detect Target Column --------
if 'y' in df.columns:
    target_col = 'y'
elif 'deposit' in df.columns:
    target_col = 'deposit'
else:
    raise Exception("Target column not found")

print("Target Column:", target_col)

df[target_col] = df[target_col].map({'yes': 1, 'no': 0})

import matplotlib.pyplot as plt
import seaborn as sns
plt.figure()
sns.countplot(x=df[target_col])
plt.title("Target Class Distribution")
plt.xlabel("Subscribed (1 = Yes, 0 = No)")
plt.ylabel("Count")
plt.show()


# -------- Preprocessing --------
df_encoded = pd.get_dummies(df, drop_first=True)

X = df_encoded.drop(target_col, axis=1)
y = df_encoded[target_col]

from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

from sklearn.linear_model import LinearRegression
       from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

y_pred_lin = lin_reg.predict(X_test)

mse = mean_squared_error(y_test, y_pred_lin)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred_lin)
r2 = r2_score(y_test, y_pred_lin)

print("\n===== Linear Regression Metrics =====")
print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)
# IMPORTANT DIAGRAM 2: Actual vs Predicted
plt.figure()
plt.scatter(y_test, y_pred_lin)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Linear Regression: Actual vs Predicted")
plt.show()


from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, confusion_matrix,
    f1_score, roc_curve, auc
)

log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train)

y_pred_log = log_reg.predict(X_test)

cm = confusion_matrix(y_test, y_pred_log)
```

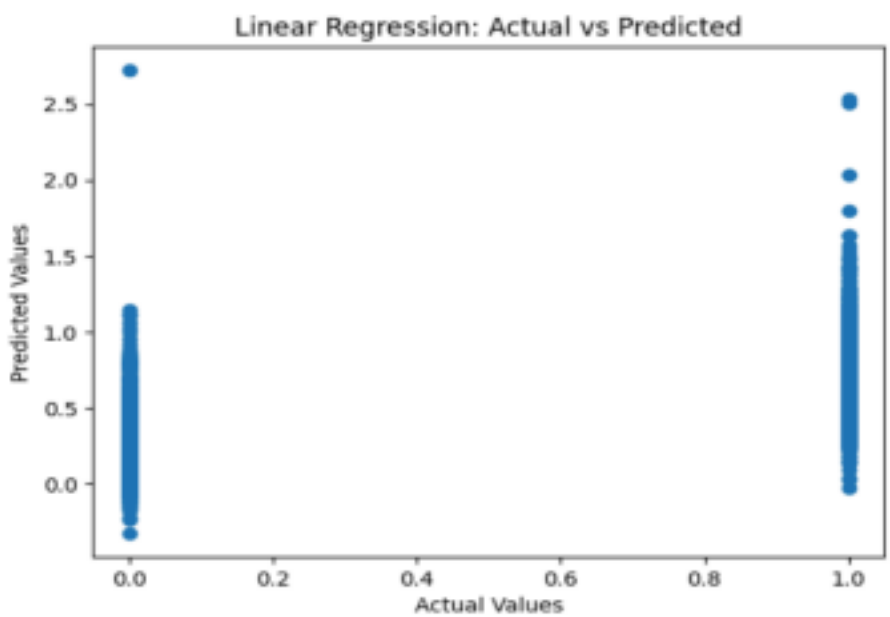
```
TN, FP, FN, TP = cm.ravel()

accuracy = accuracy_score(y_test, y_pred_log)
precision = precision_score(y_test, y_pred_log)
recall = recall_score(y_test, y_pred_log) specificity
= TN / (TN + FP)
f1 = f1_score(y_test, y_pred_log)

print("\n===== Logistic Regression Metrics =====")
print("Confusion Matrix:\n", cm)

print("\nTP:", TP)
print("TN:", TN)
print("FP:", FP)
print("FN:", FN)

print("\nAccuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("Specificity:", specificity)
print("F1 Score:", f1)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

y_prob = log_reg.predict_proba(X_test)[:,1]
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve (AUC = %0.2f)" % roc_auc)
plt.show()
```

OUTPUT:

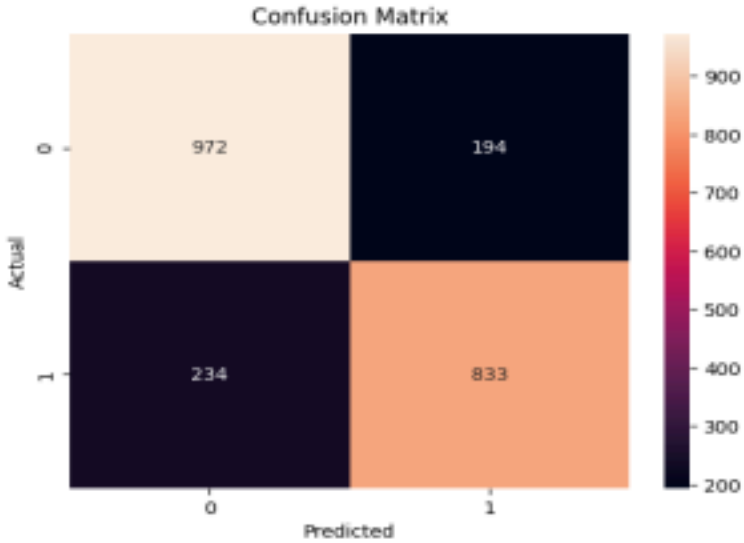
```
bank.csv(text/csv) - 910800 bytes, last modified: 2/11/2026 - 100% done
Saving bank.csv to bank (6).csv
Dataset Loaded Successfully
Columns: Index(['age', 'job', 'marital', 'education', 'default', 'balance', 'housing',
       'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays',
       'previous', 'poutcome', 'deposit'],
      dtype='object')
Target Column: deposit
```

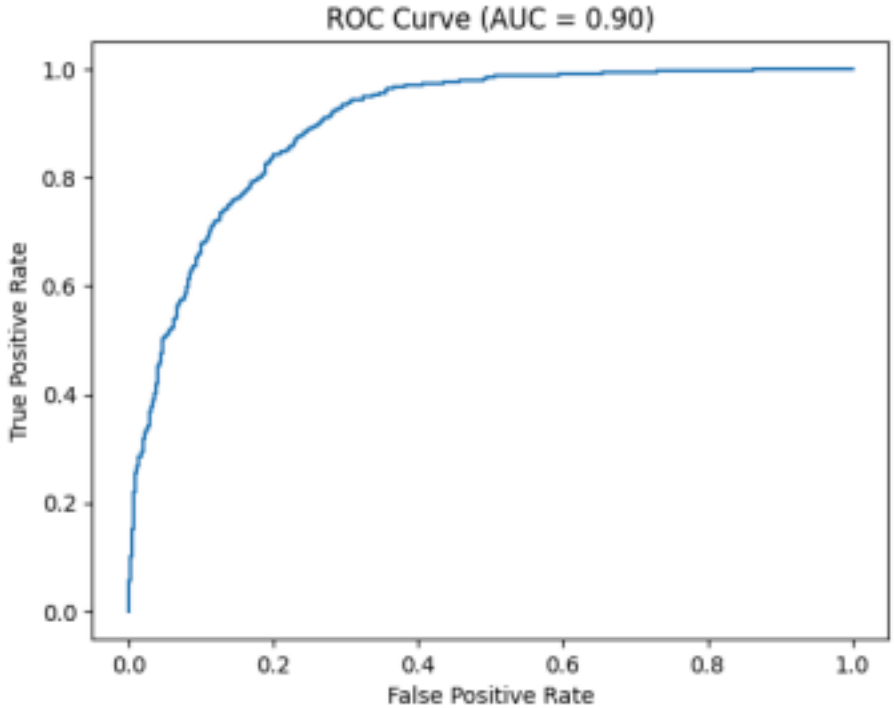



```
===== Linear Regression Metrics =====
MSE: 0.14680709131784178
RMSE: 0.3831541351960615
MAE: 0.3175254759864927
R2 Score: 0.4116151112510269
```



```
===== Logistic Regression Metrics =====
Confusion Matrix:
 [[972 194]
 [234 833]]

TP: 833
TN: 972
FP: 194
FN: 234

Accuracy: 0.8083296014330497
Precision: 0.8111002921129503
Recall: 0.7806935332708529
Specificity: 0.8336192109777015
F1 Score: 0.7956064947468959
```





Conclusion

In this experiment, Linear Regression and Logistic Regression were applied to the Bank Marketing dataset. Linear Regression was used for comparison, but it is not suitable for binary classification since it predicts continuous values. Logistic Regression performed better, as it is specifically designed for classification problems. Evaluation metrics such as Accuracy, Precision, Recall, F1-score, and the Confusion Matrix showed that Logistic Regression effectively predicts whether a customer will subscribe to a term deposit. Therefore, Logistic Regression is the more appropriate model for this problem.